

Building a Bronze → Silver → Gold ETL Pipeline

How we structured raw retail data into clean, analysis-ready tables on Databricks

1. The Big Picture

This document walks through an ETL (Extract, Transform, Load) pipeline we built on Databricks using the “medallion architecture” — a common pattern for organizing data as it moves from raw and messy to clean and business-ready. The pattern has three stages, each stored as its own set of tables:

- Bronze — raw data, landed exactly as the source sent it. Nothing is cleaned or dropped. This is our safety net: if something breaks downstream, we can always rebuild from here.
- Silver — cleaned, typed, and deduplicated data. Nested fields get flattened, bad values get handled, and each row means one clear thing (e.g. one product on one order).
- Gold — business-ready summaries. Aggregated tables built for direct use in dashboards or reports, with no further transformation needed.

Think of it like a kitchen: Bronze is the delivery of raw ingredients, Silver is the prepped and washed ingredients, and Gold is the plated dish ready to serve.

2. The Dataset

We used Databricks’ built-in sample dataset at `/databricks-datasets/retail-org` — no upload required. It contains two sources:

- `sales_orders` — JSON files, one record per order, each containing a nested list of the products in that order.
- `customers.csv` — a flat customer reference table (ID, name, state, city, loyalty segment, etc.).

3. Bronze Layer: Raw Ingestion

Each source is read as-is and written to a Delta table, with two extra columns added purely for traceability: when it was loaded (`_ingest_ts`) and which file it came from (`_source_file`). No filtering, no renaming, no type changes.

```
bronze_orders = (  
    spark.read.json(sales_path)  
    .withColumn("_ingest_ts", F.current_timestamp())  
    .withColumn("_source_file", F.input_file_name())  
)  
bronze_orders.write.format("delta").mode("overwrite") \  
    .saveAsTable("bronze.sales_orders_raw")
```

The same pattern is repeated for the customers CSV, writing to `bronze.customers_raw`.

4. Silver Layer: Cleaning & Flattening

This is where the real transformation work happens, written in SQL (CREATE OR REPLACE TABLE ... AS SELECT — the same pattern as a Snowflake CTAS). Three things happen here:

- **Flattening:** each order in Bronze contains a list of products. `explode()` turns that list into one row per product, so a single order with 3 items becomes 3 rows.
- **Type casting:** fields arrive as loosely-typed strings from JSON/CSV and get cast to proper DECIMAL, INT, DATE, and TIMESTAMP types.
- **Deduplication:** `QUALIFY ROW_NUMBER() OVER (...) = 1` keeps only the most recently loaded version of each record, in case the same order or customer appears more than once.

```
CREATE OR REPLACE TABLE silver.sales_orders AS
SELECT
  order_number, customer_id, customer_name,
  CAST(FROM_UNIXTIME(TRY_CAST(order_datetime AS BIGINT))
    AS TIMESTAMP) AS order_ts,
  product.id AS product_id,
  product.name AS product_name,
  CAST(product.price AS DECIMAL(10,2)) AS unit_price,
  CAST(product.qty AS INT) AS quantity,
  CAST(product.price AS DECIMAL(10,2))
    * CAST(product.qty AS INT) AS line_revenue
FROM (
  SELECT *, explode(ordered_products) AS product
  FROM bronze.sales_orders_raw
)
WHERE order_number IS NOT NULL
QUALIFY ROW_NUMBER() OVER (
  PARTITION BY order_number, product.id ORDER BY _ingest_ts DESC
) = 1
```

A real bug we hit — and why it matters

A handful of orders had an empty string (") in the `order_datetime` field instead of a real timestamp. Spark's strict mode refused to convert " straight to a number and the whole query failed.

Fix: wrap the conversion in `TRY_CAST(order_datetime AS BIGINT)`, which returns NULL for bad values instead of erroring out the entire job. This is a good general habit any time you cast a raw string field — real-world data always has a few dirty rows.

5. Gold Layer: Business-Ready Aggregates

Gold tables answer specific business questions directly — no joins or math required at query time. We built two:

- `daily_sales_by_state` — orders, units, and revenue grouped by day and customer state.
- `top_products` — units sold and total revenue per product, ranked highest revenue first.

```
CREATE OR REPLACE TABLE gold.top_products AS
SELECT
  product_id, product_name,
  SUM(quantity) AS units_sold,
  ROUND(SUM(line_revenue), 2) AS total_revenue
FROM silver.sales_orders
```

```
GROUP BY product_id, product_name
ORDER BY total_revenue DESC
```

6. Actual Results From This Run

Row counts across all six tables in this run:

Table	Layer	Row count
bronze.sales_orders_raw	Bronze	4,074
bronze.customers_raw	Bronze	28,813
silver.sales_orders	Silver	7,960
silver.customers	Silver	28,670
gold.daily_sales_by_state	Gold	1,770
gold.top_products	Gold	98

The shape of these numbers tells its own story: Bronze holds one row per raw order (4,074), Silver expands to one row per product line within each order (7,960, since most orders contain multiple products), and Gold compresses everything down to just 1,770 state/day summaries and 98 distinct products — exactly the kind of shrink-then-summarize pattern you'd expect from raw data down to a reporting table.

7. Lessons & Troubleshooting Notes

- Run cells top-to-bottom, in order. Several errors during this build (a “table not found” error on `bronze.customers_raw`) turned out to be because a cell simply hadn't been run yet, not a bug in the SQL itself.
- On workspaces with Unity Catalog enabled, unqualified table names resolve against whatever the current catalog session variable is — if a query switches catalogs mid-notebook, later cells can silently fail to find earlier tables.
- `TRY_CAST` over `CAST` is the safer default whenever converting raw, uncontrolled string input — it degrades to `NULL` instead of failing the whole pipeline.

8. Why This Structure Matters

Splitting the pipeline into Bronze, Silver, and Gold — rather than one big transformation — has three practical payoffs: if a transformation bug is found, Bronze lets us rebuild Silver and Gold from scratch without re-fetching source data; each layer can be tested and validated independently; and different teams can work against the layer that matches their need (engineers debug from Silver, analysts query Gold directly).

Next steps worth exploring: switching Silver/Gold from full-refresh (`CREATE OR REPLACE TABLE`) to incremental loads with `MERGE INTO`, and scheduling this pipeline to run automatically as a Databricks Job.